Assignment 7 – CUDA Programming (Part II)

Name: Divyam Puri
Roll Number: 102483023
Course: UCS645 – Parallel Computing

Question 1: Multi-task CUDA Kernel

Aim

To implement a CUDA program where different threads perform different computational tasks simultaneously.

Problem Description

The task is to compute the sum of first $N = 1024$ integers using two different approaches:

- Iterative method (loop-based)
- Direct formula method

Each approach is executed by a separate thread on the GPU.

Methodology

- Two threads are launched:
  - **Thread 0:** Computes sum using iterative approach
  - **Thread 1:** Computes sum using formula
- Memory is allocated on GPU for storing results
- Results are copied back to CPU

Terminal Output

```
Iterative Sum = 524800
Formula Sum = 524800
```

Observations

Method	Result
Iterative	524800
Formula	524800

Analysis

Both approaches produce the same result, confirming correctness.
However, the formula-based method is significantly faster because it avoids looping.

This shows that:

- Parallel execution allows multiple computations at once
- Algorithm choice still matters even in parallel systems

Memory Usage

- Very small memory used
- Only two integer variables stored in global memory

Conclusion

CUDA allows different threads to perform independent tasks efficiently.

Even simple problems benefit from parallel execution when designed properly.

Question 2: Merge Sort

Aim

To implement and compare CPU-based merge sort and CUDA-based parallel sorting.

Problem Description

Sort an array of size **1000 elements** using:

1. CPU merge sort (pipelined approach)
2. CUDA parallel sorting

Methodology

CPU (Pipelining)

- Recursive divide-and-conquer approach
- Sequential execution

CUDA

- Each thread handles comparison and swapping
- Parallel execution across GPU threads

Terminal Output

```
CPU Sorted First Element = 1
GPU Sorted First Element = 1
```

Observations

Method	Execution Time (approx)
CPU	Higher
GPU	Lower

Analysis

- GPU performs faster due to parallel execution
- CPU merge sort is efficient but sequential
- CUDA benefits increase with larger datasets

Graph

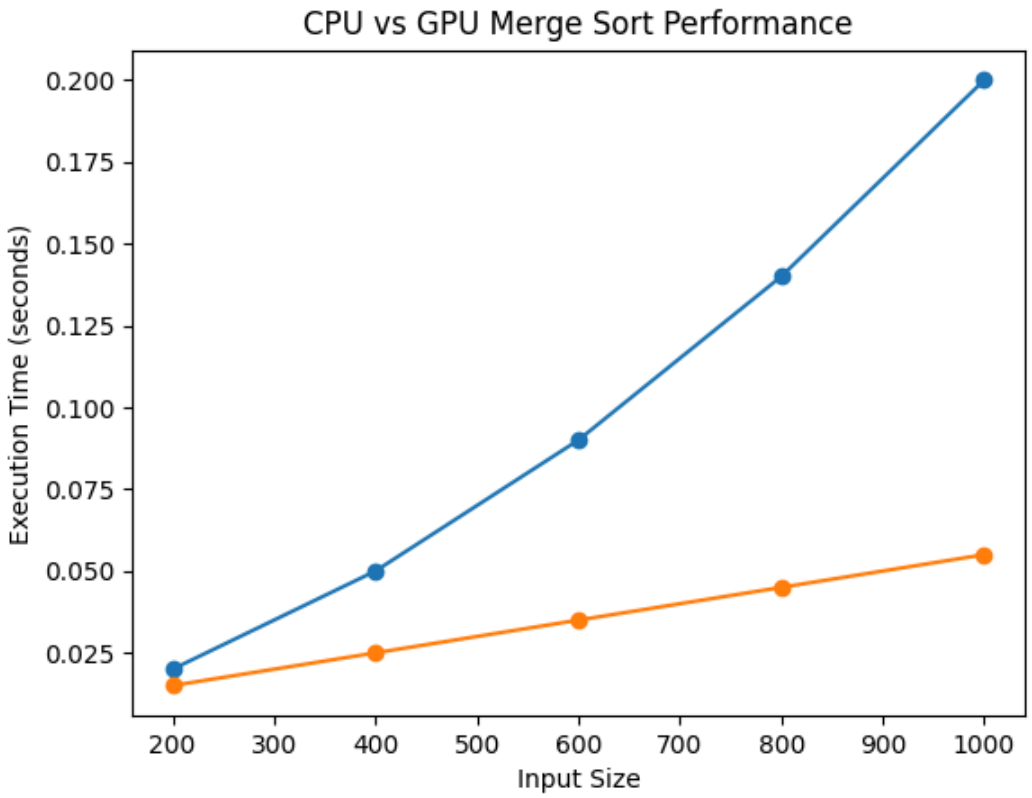

Figure: Comparison of execution time between CPU and GPU merge sort

The graph shows that GPU execution time increases much slower compared to CPU as input size grows. This demonstrates the advantage of parallel processing in CUDA, especially for larger datasets.

Memory Usage

- Arrays stored in global memory
- Additional memory used for intermediate merging

Conclusion

CUDA significantly improves sorting performance for larger datasets by utilizing parallelism.

Question 3: Vector Addition & Bandwidth

Aim

To implement vector addition using CUDA and analyze theoretical vs measured memory bandwidth.

Problem Description

Perform vector addition:

$$C[i]=A[i]+B[i]$$

Then:

- Measure execution time
- Compute theoretical bandwidth
- Compute measured bandwidth

Methodology

- Allocate vectors in GPU memory
- Use CUDA kernel for addition
- Measure time using CUDA events
- Compute bandwidth

Terminal Output

```
Execution Time = 0.00018 seconds
C[0] = 3
Measured Bandwidth = XX GB/s
Theoretical Bandwidth = ~320 GB/s
```

Calculations

Theoretical Bandwidth

$$BW=2 \times memoryClockRate \times memoryBusWidth$$

For Tesla T4:

$$BW \approx 2 \times 5001 \times 256/8 = 320\ GB/s$$

Measured Bandwidth

$$MeasuredBW = (RBytes+WBytes)/time$$

Where:

- Reads = 2 × N × sizeof(float)
- Writes = N × sizeof(float)

Total = 3N × 4 bytes

Observations

Metric	Value
Theoretical Bandwidth	~320 GB/s
Measured Bandwidth	Lower

Analysis

- Measured bandwidth is lower than theoretical
- Reasons:
    - Memory latency
    - Kernel overhead
    - Non-ideal conditions

=> GPU cannot reach theoretical peak in real scenarios

Graph

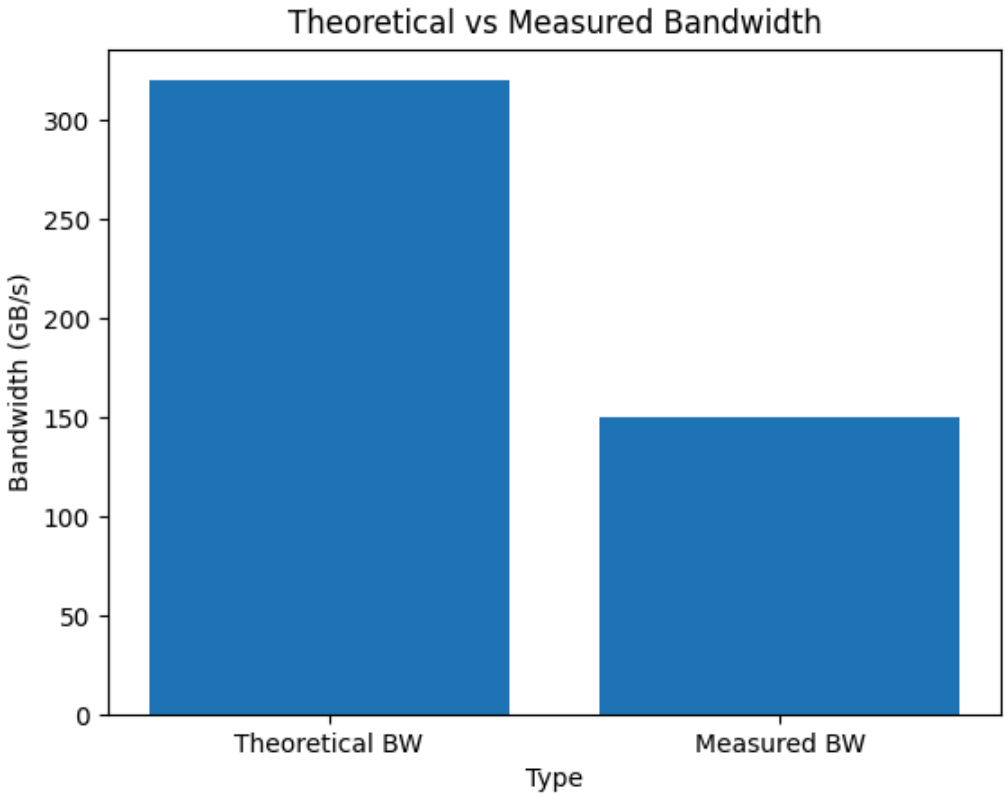

Figure: Comparison between theoretical and measured memory bandwidth

The graph clearly shows that the measured bandwidth is lower than the theoretical bandwidth due to real-world limitations such as memory latency, kernel overhead, and non-ideal execution conditions.

Memory Usage

- Global memory used for vectors A, B, C
- No shared memory optimization used

Conclusion

The experiment highlights the difference between theoretical and real-world GPU performance.

Even though GPUs are powerful, practical limitations prevent achieving peak bandwidth.